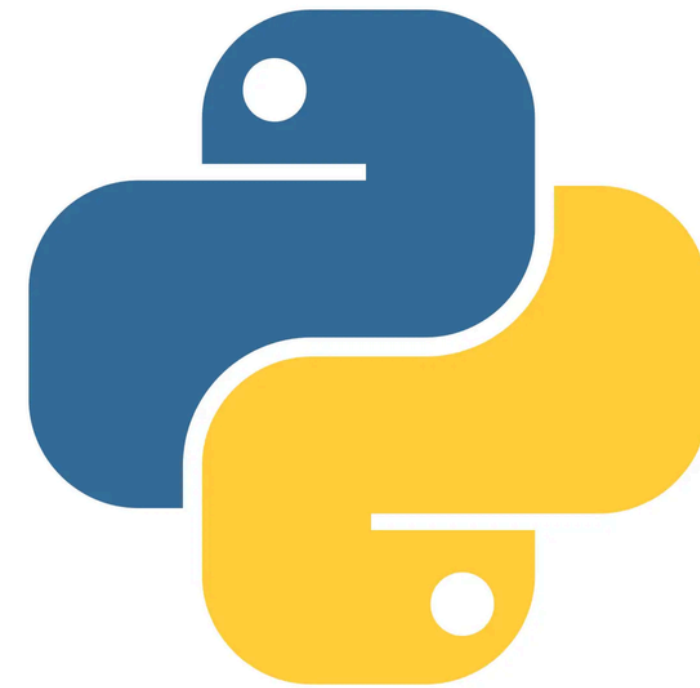


Getting Started with **Python**



Overview

- What is Python?
- Syntax
 - Indentation
 - Variables
 - Comments
- User Input
- Data Types
 - Numbers
 - Operators
 - Strings
 - Operations
- List, Tuple, Set, Dictionary

- 
- Loop
 - Function and Lambda
 - Array
 - Import Module
 - Numpy
 - Pandas
 - Matplotlib

```
self.file = None
self.fingerprints = set()
self.logdups = True
self.debug = debug
self.logger = logging.getLogger(__name__)
if path:
    self.file = open(os.path.join(path, 'fingerprint.log'), 'a')
    self.file.seek(0)
    self.fingerprints.update(self.logger.handlers)

@classmethod
def from_settings(cls, settings):
    debug = settings.getbool('debug')
    return cls(job_dir(settings), debug)

def request_seen(self, request):
    fp = self.request_fingerprint(request)
    if fp in self.fingerprints:
        return True
    self.fingerprints.add(fp)
    if self.file:
        self.file.write(fp + os.linesep)

def request_fingerprint(self, request):
    return request_fingerprint(request)
```




What is Python?

- Python is a **high-level**, **interpreted**, and **object-oriented** programming language.
- It is known for **being beginner-friendly** while remaining highly powerful and widely used in the software industry.

Syntax

Python syntax can be executed by writing directly in the Command Line:

Example:

```
▶ print("Hello World")  
⇒ Hello World
```

1) Indentation - the spaces at the beginning of a code line.

Python uses indentation to **indicate a block of code**.

```
▶ if 5 > 2:  
    print("Five is greater than two!")  
⇒ Five is greater than two!
```

Python will give you an **error** if you **skip the indentation**:

```
▶ if 5 > 2:  
    print("Five is greater than two!")  
⇒ File "<ipython-input-4-a314491c53bb>", line 2  
    print("Five is greater than two!")  
    ^  
IndentationError: expected an indented block after 'if' statement on line 1
```


2) Variables - containers for storing data values.

In Python, variables are created when you assign a value to it:

```
▶ x = 5  
y = "Hello, World!"
```

Variables **do not need** to be declared with any particular type.

```
▶ x = 4          # x is of type int  
x = "Sally"     # x is now of type str  
print(x)  
⇒ Sally
```

Casting

To specify a variable's data type in Python, you can use casting.

```
▶ x = str(3)     # x will be '3'  
y = int(3)       # y will be 3  
z = float(3)     # z will be 3.0
```

Get the type

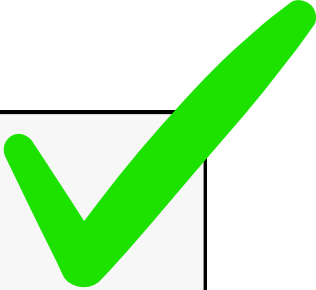
To get the data type of a variable with the **type()** function.

```
▶ x = 5  
y = "John"  
print(type(x))  
print(type(y))  
⇒ <class 'int'>  
   <class 'str'>
```

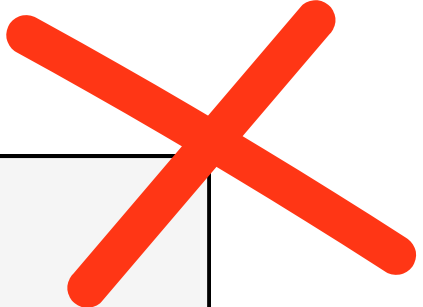
2.1) Variable Names

Rules for Python variables:

- Must start with a letter or an underscore (_).
- Cannot start with a number.
- Can only contain alphanumeric characters and underscores (A-Z, a-z, 0-9, _).
- Are case-sensitive (e.g., age, Age, and AGE are distinct).
- Cannot use Python keywords.



```
myvar = "John"  
my_var = "John"  
_my_var = "John"  
myVar = "John"  
MYVAR = "John"  
myvar2 = "John"
```



```
2myvar = "John"  
my-var = "John"  
my var = "John"
```

```
File "<ipython-input-12-0e82719f22b4>", line 1  
    2myvar = "John"  
    ^  
SyntaxError: invalid decimal literal
```


2.2) Assign Multiple Values

Many Values to Multiple Variables

Python allows you to assign values to multiple variables in one line:

```
▶ x, y, z = "Orange", "Banana", "Cherry"  
print(x)  
print(y)  
print(z)
```

```
⇒ Orange  
Banana  
Cherry
```

Unpack a Collection

To extract values from a list, tuple, or other collection into variables.

```
▶ fruits = ["apple", "banana", "cherry"]  
x, y, z = fruits  
print(x)  
print(y)  
print(z)
```

```
⇒ apple  
banana  
cherry
```

One Value to Multiple Variables

You can assign the same value to multiple variables in one line:

```
▶ x = y = z = "Orange"  
print(x)  
print(y)  
print(z)
```

```
⇒ Orange  
Orange  
Orange
```

2.3) Output Variables

The Python **print()** function is often used to output variables.

```
▶ x = "Python is awesome"  
  print(x)  
⇒ Python is awesome
```

Using the + operator to output multiple variables is allowed

```
▶ x = "Python "  
  y = "is "  
  z = "awesome"  
  print(x + y + z)  
⇒ Python is awesome
```

You output multiple variables, separated by a comma:

```
▶ x = "Python"  
  y = "is"  
  z = "awesome"  
  print(x, y, z)  
⇒ Python is awesome
```

****Combining a **string** and a **number** with the + operator will result in an **error**.**

```
▶ x = 5  
  y = "John"  
  print(x + y)  
⇒ -----  
  TypeError
```

2.4) Global Variables

- Variables that are created outside of a function.
- Global variables can be used by everyone, both inside of functions and outside.

```
▶ x = "awesome"

def myfunc():
    print("Python is " + x)

myfunc()

⇒ Python is awesome
```

```
▶ def myfunc():
    global x
    x = "fantastic"

myfunc()

print("Python is " + x)

⇒ Python is fantastic
```

The global Keyword

If you use the global keyword, the variable belongs to the global scope:

3) Comments

Comments start with a #, and Python will render the rest of the line as a comment:

```
#This is a comment.  
print("Hello, World!")
```

Multiline Comments

- Python does not really have a syntax for multiline comments.
- Use # for each line or triple quotes (""").

```
"""  
This is a comment  
written in  
more than just one line  
"""  
print("Hello, World!")
```

```
⇒ Hello, World!
```

User Input

Python allows for user input.

The method is a bit different in Python 3.6 than Python 2.7.

The following examples ask for the username, and when you entered the username, it gets printed on the screen:

```
▶ username = input("Enter username:")  
print("Username is: " + username)
```

```
↵ Enter username:Ploy  
Username is: Ploy
```

Python **3.6** uses the **input()** method.

```
username = raw_input("Enter username:")  
print("Username is: " + username)
```

```
Enter username:Ploy  
Username is: Ploy
```

Python **2.7** uses the **raw_input()** method.

Data Types - Numbers

Variables of numeric types are created when you assign a value to them:

```
▶ x = 1      # int  
  y = 2.8    # float  
  z = 1j     # complex  
  print(type(x))  
  print(type(y))  
  print(type(z))
```

```
⇒ <class 'int'>  
  <class 'float'>  
  <class 'complex'>
```

There are 3 numeric types in Python:

Integer (int)



- Numbers without a fractional part
- $a = 22$

Floating Point (float)

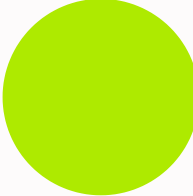


- Numbers with a decimal point
- $b = 3.14$

Complex Number (complex)



- Numbers with imaginary parts
 $c = 1 + 2j$ (real part + imaginary part)



Operations for Numbers

- Normal mathematical operations, for example:
 - Addition (+)
 - Subtraction (-)
 - Multiplication (*), Power (**)
 - Division (/), Floor division (//)
- Built-in mathematical functions, for example:
 - **pow, abs, round, int, hex, bin**
- Utility modules, for example:
 - **import math, import random**

Arithmetic Operators

Operator	Name	Description	Example
+	Addition	Adding two variables/values	$x + y$
-	Subtraction	Subtracting two variables/values	$x - y$
*	Multiplication	Multiple two variables/values	$x * y$
/	Division	Divide two variables/values (Numerator / Denominator)	x / y
//	Floor division	The operation that rounds down the result of a division operation to the nearest integer	$x // y$
%	Modulus	The remainder of the division of one number by another	$x \% y$
**	Power or exponentiation	The operation of raising one quantity to the power of another	$x ** y$

Some operators can do more

Operator	Name	Description	Example
+	Addition, Concatenation	Adding two variables/values Concatenate lists, strings	$x + y$
-	Subtraction, Set difference	Subtracting two variables/values Finding difference between two sets	$x - y$

Logical Operators (cont.)

Operator	Name	Description	Example
<code>or</code>	Logical OR	Evaluates to true if <i>at least one</i> of the conditions is true	<code>x or y</code>
<code>and</code>	Logical AND	Evaluates to true if <i>all</i> conditions are true	<code>x and y</code>
<code>not</code>	Logical NOT	Reverses the logical state of its operand	<code>not x</code>

Relational Operators

Operator	Name	Description	Example
<	Is less than	Compare if one variable/value is less than another	x < y
<=	Is less than or equal to	Compare if one variable/value is less than or equal to another	x <= y
>	Is greater than	Compare if one variable/value is greater than another	x > y
>=	Is greater than or equal to	Compare if one variable/value is greater than or equal to another	x >= y
==	Is equal to	Compare if one variable/value equals to another	x == y
!=	Is not equal to	Compare if one variable/value differs from another	x != y

Data Types - Strings

- Strings are sequences of one-character strings
'spam'
- Other types of sequences include lists and tuples
- Sequence operations
 - Get Length: **len('spam')**
 - Indexing:



S = **'spam'**



S[0]

The first item in S



S[1]

The second item
from the left

Operations in Strings

- Indexing (cont.)

S = 'spam'

S[-1] # The last item from the end in S

S[-2] # The second to last item from the end

S[len(S)-1] # Negative indexing, the hard way

- Slicing

S[1:3] # Slice of S from offsets 1 through 2 (not 3)

S[1:] # Everything past the first (1:len(S))

S[0:3] # Everything but the last

Syntax

S[start:end]

Operations in Strings (cont.)

- Slicing (cont.)

S[:3] # Same as S[0:3]

S[:-1] # Everything but the last again

S[:] # All of S as a top-level copy (0:len(S))

- Concatenation

S + 'xyz' # Check out the result

- Repetition

S * 8 # Check out the result

Immutability

- Every string operation is defined to produce a new string as its result
- Original is not changed

S = 'spam'

S[0] = 'z' # Does this work?

TypeError: 'str' object does not support item assignment

S = 'z' + S[1:] # How about this?

More Methods for Strings

S = 'spam'

- Search substring

S.find('pa') # Find the offset of a substring

➡ 1

- Search and replace

S.replace('pa','crea') # Replace 'pa' with 'crea'

➡ 'scream'

- Split string

line = 'aaa,bbb,ccccc,dd'

line.split(',') # Check out the result

More Methods for Strings

S = 'spam'

- Convert cases

S.upper() # Upper- and lowercase conversions

➡ **SPAM**

- Test string content

S.isalpha() # Content tests: isalpha, isdigit, etc.

➡ **True**

- Strip whitespace off the ends of the string

line = 'aaa,bbb,ccccc,dd\n'

line.rstrip() # Remove whitespace on the right side

➡ **'aaa,bbb,ccccc,dd'**



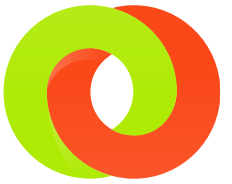
List

- The most general sequence
- Is a positionally ordered collection of arbitrarily typed objects
- Has no fixed size
- Mutable = can be modified in-place by assignment
- Allows duplicates

Syntax

`my_list = [element1, element2, ...]`





List

Empty list

```
my_list = [ ]
```

List of same-type elements

```
my_list_int = [1, 2, 3, 4]
```

```
my_list_str = ['one', 'two', 'three', 'four']
```

```
my_list_bool = [True, False, False, True]
```



List of mixed-type elements

```
my_list_mixed = [1, 'two', 3.0, True]
```



Access List

```
my_list = [1, 2, 3, 4, 5]
```

- Indexing

```
first_elem = my_list[0] ➡ 1
```

- Negative Indexing

```
last_elem = my_list[-1] ➡ 5
```

- Slicing

```
sub_list = my_list[1:3] ➡ [2, 3]
```

Modify List (cont.)

```
mylist = [1, 'two', 3, 4, 5, 20, 21, 22]
```

- Remove element from a list

```
mylist.remove('two')
```

➡ [1, 3, 4, 5, 20, 21, 22]

- Clear a list

```
mylist.clear()
```

➡ []

- Pop element off a list (i.e. another way to remove)

```
mylist.pop() # remove last elem
```

➡ [1, 3, 4, 5, 20, 21]

```
mylist.pop(1) # remove indexed elem
```

➡ [1, 4, 5, 20, 21]

Common Methods in List

```
mylist = [1, 2, 0, 9, 7, 8]
```

- Sort

```
mylist.sort()
```

👉 [0, 1, 2, 7, 8, 9]

```
mylist.sort(reverse=True)
```

👉 [9, 8, 7, 2, 1, 0]

- Reverse

```
mylist.reverse()
```

- Index

```
id = mylist.index(2)
```

👉 3

- Count

```
count = mylist.count(9)
```

👉 1

More Operations in List

```
mylist = [9, 8, 7, 2, 1, 0]
```

- Get length

```
len(mylist)
```

➡ 6

- Concatenate lists

```
list1 = [1, 2]
```

```
list2 = [3, 4]
```

```
list1 + list2
```

➡ [1, 2, 3, 4]

- Repetition

```
list1 * 3
```

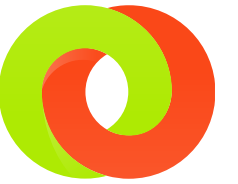
➡ [1, 2, 1, 2, 1, 2]

- Membership

```
2 in list1
```

➡ True

Dictionary



- Is an **unordered**, **mutable** collection of key-value pairs
- Not a sequence, instead it is a **mapping**!
 - Store objects by key instead of by relative position
 - Map keys to associated values

Syntax

```
my_dict = {"key1": "value1", "key2": "value2"}
```

Empty dictionary

```
my_dict = {}
```

Dictionary with elements

```
student = {"name": "Somsak", "age": 25}
```

Access Dictionary

```
student = {"name": "Somsak", "age": 25}
```

- By key

```
student["name"]
```

➡ "Somsak"

- Through a method .get(...)

```
student.get("name")
```

➡ 'Somsak'

```
student.get("age")
```

➡ 25

Can we do this? What's the output?

```
student = {  
    "name": "Somsak",  
    "age": 25,  
    "age": 28  
}
```

Modify Dictionary

```
student = {"name": "Somsak", "age": 25}
```

- Change value

```
student["name"] = "Kraisak"
```

```
student.update({"age": 40})
```

- Add items

```
student["country"] = "Thailand"
```

- Remove items

```
student.pop("country")
```

```
student.popitem() # last inserted item
```

```
del student["age"]
```

```
del student # delete dict
```

```
student.clear() # remove all elements
```


Common Methods in Dictionary

```
student = {"name": "Somsak", "age": 25}
```

- Get keys

```
student.keys()
```

👉 `dict_keys(['name', 'age'])`

- Get values

```
student.values()
```

👉 `dict_values(['Somsak', 25])`

```
student.get("name")
```

👉 `'Somsak'`

- Copy a dictionary

```
student.copy()
```

- Get items (as a tuple)

```
student.items()
```

👉 `dict_items([('name', 'Somsak'), ('age', 25)])`

More Operations in Dictionary

```
student = {"name": "Somsak", "age": 25}
```

- Merge dictionaries

```
d1 = {'a': 1, 'b': 2}
```

```
d2 = {'b': 3, 'c': 4}
```

```
d1.update(d2)
```

- Or use | operator (Python 3.9+)

```
d1 | d2
```

- Get length

```
len(student)
```

👉 2

- Check for keys

```
"name" in student
```

👉 True

Tuple

- Is an **immutable, ordered** collection of elements
- Defined by enclosing the elements in parentheses ()
- Elements in a tuple can be of any data type
- Is used to **group related data** and **cannot be modified** after creation
- Allow duplicates

Syntax

```
my_tuple = (element1, element2, ...)
```

Tuple

Empty tuple

```
my_tuple = ()
```

Tuple of single element

```
my_tuple = (1,)
```

Tuple of multiple elements

```
my_tuple = (1, 2, 3)
```

```
my_tuple = 1, 2, 3 # without parenthesis
```

```
my_tuple = 1, 'two', 3 # mixed-type elements
```

Access Tuple

```
my_tuple = (1, 2, 3, 4, 5)
```

- Indexing

```
first_elem = my_tuple[0]
```

👉 1

- Negative Indexing

```
last_elem = my_tuple[-1]
```

👉 5

- Slicing

```
sub_tuple = my_tuple[1:3]
```

👉 (2, 3)

Modify Tuple (Can we?)

```
my_tuple = (1, 2, 3, 4, 5)
```

- Change value (workaround)

```
temp = list(my_tuple)
```

```
temp[1] = "two"
```

```
my_tuple = tuple(temp)
```

- Add item (workaround #1)

```
temp = list(my_tuple)
```

```
temp.append("orange")
```

```
my_tuple = tuple(temp)
```

- Add item (workaround #2)

```
my_tuple_2 = ("orange",)
```

```
my_tuple += my_tuple_2
```


Common Methods in Tuple

```
my_tuple = (1, 2, 3, 4, 5, 5)
```

- Index

```
my_tuple.index(2)
```

👉 1

- Count

```
my_tuple.count(5)
```

👉 2

Operations in Tuple

```
my_tuple = (1, 2, 3, 4, 5, 5)
```

- Get length

```
my_tuple = (1, 2, 3, 4, 5, 5)
```

```
len(my_tuple)
```

➡ 6

- Concatenate tuples

```
tuple1 = (1,2,3)
```

```
tuple2 = (4,5,6)
```

```
tuple1 + tuple2
```

➡ (1,2,3,4,5,6)

- Membership

```
3 in tuple1
```

➡ True

- Repetition

```
tuple1 * 2
```

➡ (1,2,3,1,2,3)

- Get max

```
max(my_tuple)
```

➡ 5

- Get min

```
min(my_tuple)
```

➡ 1

- Unpacking

```
tuple1 = (1,2,3)
```

```
a, b, c = tuple_1
```


Set

- Is an **unordered** collection of **unique** elements
- Is **mutable**
- Do not allow duplicate elements
- Is commonly used for membership testing and eliminating duplicates

Syntax

```
my_set = {element1, element2, ...}
```

Empty set

```
my_set = set()
```

Non-empty set

```
my_set = {1, 2, 3}
```

```
my_set = set([1, 2, 3, 4])
```

```
my_set = {"apple", "banana", "cherry", True, 2}
```

However, True & 1, and False & 0 are considered the same value.

```
my_set = {"apple", "banana", "cherry", True, 1, 2}
```

```
👉 {True, 2, 'banana', 'cherry', 'apple'}
```

Access Set

```
my_set = {1, 2, 3, 4, 5}
```

- Sets are unordered, no indexing or slicing
- Cannot access items in a set by referring to an index or a key

```
for element in my_set:  
    print(element)
```

Common Methods in Set

```
my_set = {1, 2, 3, 4, 5}
```

Add element

```
my_set.add(6)
```

➡ {1, 2, 3, 4, 5, 6}

```
my_set.add(2)
```

➡ {1, 2, 3, 4, 5, 6}

```
temp = {7,8}
```

```
my_set.update(temp)
```

➡ {1, 2, 3, 4, 5, 6, 7, 8}

Remove element

```
my_set.discard(6)
```

➡ {1, 2, 3, 4, 5, 7, 8}

```
my_set.discard(6)
```

➡ {1, 2, 3, 4, 5, 7, 8}

```
my_set.remove(5)
```

➡ {1, 2, 3, 4, 7, 8}

```
my_set.remove(5)
```

➡ Traceback (most recent call last):
File "<stdin>", line 1, in <module>
KeyError: 5

Common Methods in Set

```
my_set = {1, 2, 3, 4, 7, 8}
```

Clear all items in the set

```
my_set.clear()
```

👉 `set()`

Delete set entirely

```
del my_set
```

👉 `my_set`

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

NameError: name 'my_set' is not defined

Operations in Set

Union

`set1 | set2`

Intersection

`set1 & set2`

Difference

`set1 - set2`

Subset

`set1 <= set2`

Superset

`set1 >= set2`

Membership

`5 in set1`

`5 not in set1`



Control Flow Statement

```
__init__(self):  
    gpu = gpuInfo.get_gpu(0)  
    self.  
    self.  
    self.  
    self.power = gpu.query_power()  
    self.voltage = round(gpu.query_graphics_voltages()[0])  
    fans = sensors_fans()  
    for name, value in fans.items():  
        setattr(self, name, value[0][1])
```


Don't Forget “Indentation”

Python relies on **INDENTATION** to define scope in the code.

Example

```
a = 33
b = 200
if b > a:
    print("b is greater than a") # you will get an error
ⓧ 0.0s
```

Cell In[1], line 4

```
    print("b is greater than a") # you will get an error
    ^
IndentationError: expected an indented block after 'if' statement on line 3
```

Indentation

(noun) “the spaces at the beginning of a code line”

If Statement

“The code block will be execute whether condition is **TRUE** ”

```
if condition:  
    # Code to execute if the condition is True
```

Example

```
a = 33  
b = 200  
if b > a:  
    print("b is greater than a")
```

✓ 0.0s

b is greater than a

```
number = 15
```

```
if number > 0:  
    print(f"{number} is a positive number")
```

✓ 0.0s

15 is a positive number



Elif Statement

the keyword to say:

"if the previous conditions were not true, then try this condition"

```
if condition1:  
    # Code to execute if condition1 is True  
elif condition2:  
    # Code to execute if condition2 is True  
elif condition3:  
    # Code to execute if condition3 is True
```

Example

```
a = 33
b = 33
if b > a:
    print("b is greater than a")
elif a == b:
    print("a and b are equal")
```

✓ 0.0s

a and b are equal

```
x = 7
if x < 5:
    print("x is less than 5")
elif x == 7:
    print("x is equal to 7")
else:
    print("x is greater than 5 but not equal to 7")
```

✓ 0.0s

x is equal to 7

If-else Statement

The **else** part provides an alternative block of code to execute if the if condition is **False**.

if condition:

Code to execute if the condition is True

else:

Code to execute if the condition is False

Example

```
x = 3
if x > 5:
    print("x is greater than 5")
else:
    print("x is not greater than 5")
```

✓ 0.0s

x is not greater than 5

```
number = 17

if number % 2 == 0:
    print(f"{number} is an even number")
else:
    print(f"{number} is an odd number")
```

✓ 0.0s

17 is an odd number

Nested If statement

You can have if statements inside if statements, this is called **nested if statements**.

Example

```
x = 41

if x > 10:
    print("Above ten,")
    if x > 20:
        print("and also above 20!")
    else:
        print("but not above 20.")
```

✓ 0.0s

```
Above ten,
and also above 20!
```

The pass Statement

It's essentially a "**do nothing**" operation that allows your code to remain syntactically correct (**some statements can't be empty**).

```
x = 10

if x > 5:
    pass # Placeholder for future code
else:
    print("x is 5 or less")
```

✓ 0.0s

```
def my_function():
    pass # Function stub
```

✓ 0.0s

match-case Statement

Provides pattern matching functionality, similar to **a switch-case**

```
traffic_light = "yellow"

match traffic_light:
    case "red":
        print("Stop")
    case "yellow":
        print("Prepare to stop")
    case "green":
        print("Go")
    case _:
        print("Invalid traffic light color")
```

✓ 0.0s

Prepare to stop

Loop Control

```
__init__(self):
    gpu = gpuInfo.get_gpu(0)
    self.
    self.
    self.
    self.
    self.power = gpu.query_power()
    self.voltage = round(gpu.query_graphics_
    fans = sensors_fans()
    for name, value in fans.items():
        setattr(self, name, value[0][1])
```

Primitive Loop in Python

While Loop

```
while condition:  
    # Code to execute while the condition is True
```

For Loop

```
for variable in sequence:  
    # Code to execute for each item
```

While Loops

With the **while** loop, we can execute a set of statements as long **as a condition is true**.

```
count = 0
while count < 5:
    print(count)
    count += 1 # Increment count
```

✓ 0.0s

0
1
2
3
4

The break Statement

With the **break** statement we can **stop the loop** even if the while condition is true:

```
i = 1
while i < 6:
    print(i)
    if i == 3:
        break
    i += 1
```

✓ 0.0s

1
2
3

The continue Statement

Skip the rest of the code in the current iteration and **move directly** to the next iteration of the loop.

```
num = 0
while num < 10:
    num += 1
    if num % 2 == 0:
        continue # Skip the rest of the loop for even numbers
    print(num)
```

✓ 0.0s

1
3
5
7
9

For loops

The **for loop** in Python iterates **over a sequence (like a list, string, tuple, dictionary, or range)** and executes a block of code for each item in the sequence.

```
for x in "banana":  
    print(x)
```

✓ 0.0s

b
a
n
a
n
a

```
for i in range(10):  
    if i == 5:  
        break  
    print(i)
```

✓ 0.0s

0
1
2
3
4

```
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    print(x)
```

✓ 0.0s

apple
banana
cherry

```
for x in range(2, 30, 3):  
    print(x)
```

✓ 0.0s

2
5
8
11
14
17
20
23
26
29

Else Statement

We can use the **else** statement after a loop **is completed** or when the condition **no longer is true**.

```
count = 0
while count < 3:
    print(count)
    count += 1
else:
    print("While loop finished.")
```

✓ 0.0s

```
0
1
2
While loop finished.
```

Nested Loops

A nested loop **is a loop inside a loop**.

The **"inner loop"** will be executed one time for each iteration of the **"outer loop"**:

```
for i in range(1, 4): # Outer loop
    j = 1
    while j <= 3: # Inner loop
        print(f"Outer: {i}, Inner: {j}")
        j += 1
```

✓ 0.0s

```
Outer: 1, Inner: 1
Outer: 1, Inner: 2
Outer: 1, Inner: 3
Outer: 2, Inner: 1
Outer: 2, Inner: 2
Outer: 2, Inner: 3
Outer: 3, Inner: 1
Outer: 3, Inner: 2
Outer: 3, Inner: 3
```

```
rows = 5
```

```
for i in range(1, rows + 1): # Outer loop for rows
    for j in range(1, i + 1): # Inner loop for columns
        print("*", end="") # Print stars without a newline
    print() # Move to the next line after each row
```

✓ 0.0s

```
*
**
***
****
*****
```


Function

```
class GPUInfo:
    def __init__(self):
        gpu = gpuInfo.get_gpu(0)
        self.gpu = gpu
        self.power = gpu.query_power()
        self.voltage = round(gpu.query_graphics_voltage())
        fans = sensors_fans()
        for name, value in fans.items():
            setattr(self, name, value[0][1])
```



Creating a Function

A function is **a reusable block of code** that performs a specific task, helping organize your program into smaller, manageable pieces.

```
def function_name(parameters):  
    # Code block  
    return value # (Optional)
```



Example

Simple Function

```
def greet():  
    print("Hello, welcome to Python!")  
  
greet()  
✓ 0.0s  
Hello, welcome to Python!
```

Function with Parameters

```
def greet(name):  
    print(f"Hello, {name}!")  
  
greet("Joe")  
✓ 0.0s  
Hello, Joe!
```

```
def add(a, b):  
    return a + b  
  
result = add(2,1)  
print(result)  
✓ 0.0s  
3
```

Default Argument

```
def greet(name="Guest"):  
    print(f"Hello, {name}!")  
  
greet()  
greet("Joe")  
✓ 0.0s  
Hello, Guest!  
Hello, Joe!
```

Variable-length

```
def add(*args):  
    return sum(args)  
  
result = add(1, 2, 3, 4)  
print(result) # Output: 10  
✓ 0.0s  
10
```

Recursive

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n - 1)  
  
result = factorial(5)  
print(result) # Output: 120  
✓ 0.0s  
120
```



Lambda Function

A lambda function is **a small anonymous function**.

A lambda function can take any number of arguments, but can **only have one expression**.

```
lambda arguments : expression
```



Example

Multiply function

```
multiply = lambda x, y: x * y  
print(multiply(5, 4)) # Output: 12
```

✓ 0.0s

20

Implement function with lambda function(1)

```
def myfunc(n):  
    return lambda a : a * n
```

```
mydoubler = myfunc(2)  
mytripler = myfunc(3)
```

```
print(mydoubler(11))  
print(mytripler(11))
```

✓ 0.0s

22

33

Implement function with lambda function(2)

```
# Define a function that takes a parameter 'n'  
def myfunc(n):  
    # Return a lambda function that takes a parameter 'a' and multiplies it by 'n'  
    return lambda a: a * n # The lambda function remembers the value of 'n' from the enclosing scope
```

```
# Call 'myfunc' with 'n' set to 2 and assign the returned lambda function to 'mydoubler'  
mydoubler = myfunc(2)
```

```
# Call 'mydoubler' with 'a' set to 11, and it will multiply 11 by 2 (the 'n' value remembered from 'myfunc')  
print(mydoubler(11)) # Output: 22
```

✓ 0.0s

22



When do we use Lambda function?

- For **small, simple, one-liner** functions.
- When you need an anonymous function and **don't want to define a full function.**
- In scenarios where the function is **used only once or twice**, reducing the need for full function definitions.

File Handling

```
__init__(self):  
    gpu = gpuInfo.get_gpu(0)  
    self.  
    self.  
    self.  
    self.power = gpu.query_power()  
    self.voltage = round(gpu.query_graphics_voltages()[0])  
    fans = sensors_fans()  
    for name, value in fans.items():  
        setattr(self, name, value[0][1])
```



File Handling

The process of **creating, reading, updating, and deleting files** in Python

- **File types:**

- Text file (.txt)
- Binary file (.bin)
- Binary text file (.dat)
 - Stores data that is not just plain text
 - Is not always printable on the screen
- Excel file (.xlsx)
- CSV file (.csv) is short for Comma-Separated Values.
- JSON file (.json) short for JavaScript Object Notation.
- Pickle file (.pkl)
 - Is native to Python
 - Is used to serialize and deserialize data types
 - Save data such as lists, dictionaries, or other Python data structures



persons.xlsx - Saved

	A	B	C
1	Family Name	Given Name	VIAF ID
2	Ackersdijck	Willem Cornelis	17959345
3	Adelung	Friedrich von	22963658
4	Afzelius	Arvid August	49972119
5	Amerling	Karel	13331054
6	Anton	Karl Gottlob von	183632821
7	Arwidsson	Adolf Ivar	8184878
8	Asbjørnsen	Peter Christen	116587918
9	Attems	Heinrich	37665468
10	Atterbom	Per Daniel Amadeus	46819248
11	Balabin	Viktor Petrovich	44473845
12	Banks	Joseph	46830189
13	Beck	Friedrich	44338671
14	Becker	Reinhold von	42101066
15	Bernhart	Johann Baptist	69674335
16	Bertram	Johann	32890043

Excel file

persons.csv - Notepad

```
File Edit Format View Help
Family Name,Given Name,VIAF ID
Ackersdijck,Willem Cornelis,17959345
Adelung,Friedrich von,22963658
Afzelius,Arvid August,49972119
Amerling,Karel,13331054
Anton,Karl Gottlob von,183632821
Arwidsson,Adolf Ivar,8184878
Asbjørnsen,Peter Christen,116587918
Attems,Heinrich,37665468
Atterbom,Per Daniel Amadeus,46819248
Balabin,Viktor Petrovich,44473845
Banks,Joseph,46830189
Beck,Friedrich,44338671
Becker,Reinhold von,42101066
Bernhart,Johann Baptist,69674335
Bertram,Johann,32890043
Bilderdijk,Willem,14882166
Boisserée,Sulpiz,7483155
Bopp,Franz,61614118
Borovský,Karel Havlíček,100277614
Bosković,Jovan,161354270
Buslaev,Fyodor,10074560
Cenowa,Florian Stanislaw,44466031
Chomiakov,Aleksei,66492873
```

CSV file



```
- {  
  userId: 1,  
  id: 2,  
  title: "quis ut nam facilis et officia qui",  
  completed: false  
},  
- {  
  userId: 1,  
  id: 3,  
  title: "fugiat veniam minus",  
  completed: false  
},  
- {  
  userId: 1,  
  id: 4,  
  title: "et porro tempora",  
  completed: true  
},  
- {  
  userId: 1,  
  id: 5,  
  title: "laboriosam mollitia et enim quasi adipisci quia provident illum",  
  completed: false  
},
```

```
{  
  "orders": [  
    {  
      "orderno": "748745375",  
      "date": "June 30, 2088 1:54:23 AM",  
      "trackingno": "TN0039291",  
      "custid": "11045",  
      "customer": [  
        {  
          "custid": "11045",  
          "fname": "Sue",  
          "lname": "Hatfield",  
          "address": "1409 Silver Street",  
          "city": "Ashland",  
          "state": "NE",  
          "zip": "68003"  
        }  
      ]  
    }  
  ]  
}
```

JSON file



File Handling **Modes**

Operation	Mode	Description
Read	r, rt	Default mode. Open text file for reading.
Write	w, wt	Open the file for writing. If the file already exists, it will be overwritten.
Append	a, at	Open the file for writing. Insert data at the end of the file.
Create	x, xt	Open the file for writing while making sure that the file does not exist yet



File Handling **Modes (r+ w+ a+ x+)**

Operation	Mode	Description
Read	r+	Open the file for reading and writing.
Write	w+	Open the file for reading and writing. If the file already exists, it will be overwritten.
Append	a+	Open the file for reading and writing. Insert data at the end of the file.
Create	x+	Open the file for reading and writing while making sure that the file does not exist yet

‘+’ supports read & write at the same time.



File Handling **Modes**

Features	r	w	a	x
Allows read	✓			
Allows write		✓	✓	✓
Must exist	✓			
Must not exist				✓
Positioned at start	✓	✓		✓
Positioned at end			✓	
Always writes to end			✓	



File Handling Modes '+'

Features	r+	w+	a+	x+
Allows read	✓	✓	✓	✓
Allows write	✓	✓	✓	✓
Must exist	✓			
Must not exist				✓
Truncates (clear file)		✓		
Positioned at start	✓	✓		✓
Positioned at end			✓	
Always writes to end			✓	



Open File

Syntax

```
file = open('filename.txt', 'mode')
```

Example

```
file = open('example_cpe232.txt', 'r')
```

Remarks

```
file.close()
```

Could be a pain in the neck if you forget to close a file.



Open File (Cont.)

Syntax

```
with open('filename.txt', 'mode') as some_name:
```

Example

```
with open('example_cpe232.txt', 'r') as f:
```

```
with open('example_cpe232.txt', mode='r') as f:
```




Read a File

Three methods to read a file:

1. **read()** → reads the entire file as a string
2. **readline()** → reads one line at a time
3. **readlines()** → reads all lines and returns them
as a list



Write to a File

Overwrite a file

```
with open('example_cpe232.txt', 'w') as f:  
    f.write('Overwrite the whole file..')
```

Append to a file

```
with open('example_cpe232.txt', 'a') as f:  
    f.write('Just append this line..')
```

Create a new file

```
with open('brand_new_file.txt', 'x') as f:  
    f.write('Create a new one..')
```



Read & Write to a File

Read first, then write

```
with open('example_cpe232.txt', 'r+') as f:  
    content = f.read()  
    f.write('\nNow I want to write a little more.')
```

Write new content at the beginning, followed by previous content

```
with open('example_cpe102.txt', 'r+') as f:  
    original = f.read()  
    f.seek(0)  
    f.write('Write this line first.\n\n')  
    f.write(original)
```



CSV File: Read

- Comma-separated value
- Python has a built-in module to handle csv file type → import csv

size

Size	Abbreviated Size
Extra Small	XS
Small	S
Medium	M
Large	L
Extra Large	XL

```
import csv

# Open the CSV file
with open('size.csv', mode='r') as file:
    # Create a CSV reader object
    csv_reader = csv.reader(file)

    # Iterate over each row
    for row in csv_reader:
        print(row)
```

csv.reader(file): Creates a reader object that reads each row as a list of values.

['Size', 'Abbreviated Size']
['Extra Small', 'XS']
['Small', 'S']
['Medium', 'M']
['Large', 'L']
['Extra Large', 'XL']



CSV File: Write

- Relatively similar to a text file
- Data in a list format

```
students_data = [  
    ["Name", "Age", "Grade"],  
    ["Alice", 21, "A"],  
    ["Bob", 22, "B"],  
    ["Charlie", 23, "C"]  
]
```

```
with open('sample_csv_students.csv', mode='w', newline='') as file:  
    writer = csv.writer(file)  
    writer.writerows(students_data)
```

- Data in a dictionary format

```
students_data = [  
    {"Name": "Alice", "Age": 21, "Grade": "A"},  
    {"Name": "Bob", "Age": 22, "Grade": "B"},  
    {"Name": "Charlie", "Age": 23, "Grade": "C"},  
]
```

```
writer = csv.DictWriter(file, fieldnames=  
    ["Name", "Age", "Grade"])  
writer.writeheader()  
writer.writerows(students_data)
```




JSON File: Read

- JavaScript Object Notation
- Python has a built-in module to handle JSON file type → **import json**

```
import json

# Open and read the JSON file
with open('sample_json_apple.json', 'r') as file:
    data = json.load(file)

# Print the entire data
print(data)
```

json.load(file): Parses the JSON data and converts it into a Python dictionary.

```
{
    "fruit": "Apple",
    "size": "Large",
    "color": "Red"
}
```

{'fruit': 'Apple', 'size': 'Large', 'color': 'Red'}



JSON File: Write

- Use `json.dump()`
- Give a filename, the output will be written to this file.

```
data = {  
    "name": "Bob",  
    "age": 25,  
    "city": "Los Angeles",  
    "hobbies": ["surfing", "gaming"]  
}  
  
# Write data to a JSON file  
with open('sample_json_write.json', 'w') as file:  
    json.dump(data, file)
```

Import a Module

```
class _GpuInfo:
    def __init__(self):
        gpu = gpuInfo.get_gpu(0)
        self.power = gpu.query_power()
        self.voltage = round(gpu.query_graphics_voltage())
        fans = sensors_fans()
        for name, value in fans.items():
            setattr(self, name, value[0][1])
```


How to import a module?

Import the whole module

```
import math

result = math.sqrt(16)
print(result)
```

✓ 0.0s

4.0

Import specific components

```
from math import sqrt

result = sqrt(16)
print(result)
```

✓ 0.0s

4.0

Import with aliases (you name it.)

```
import math as cpe

result = cpe.sqrt(16)
print(result)
```

✓ 0.0s

4.0

Import a custom module

Create a Python file: Write your functions, classes, or variables in a Python file.
This file will act as your module.

```
# mymodule.py
def greet(name):
    return f"Hello, {name}!"

def add(a, b):
    return a + b
```

✓ 0.0s

Import a custom module

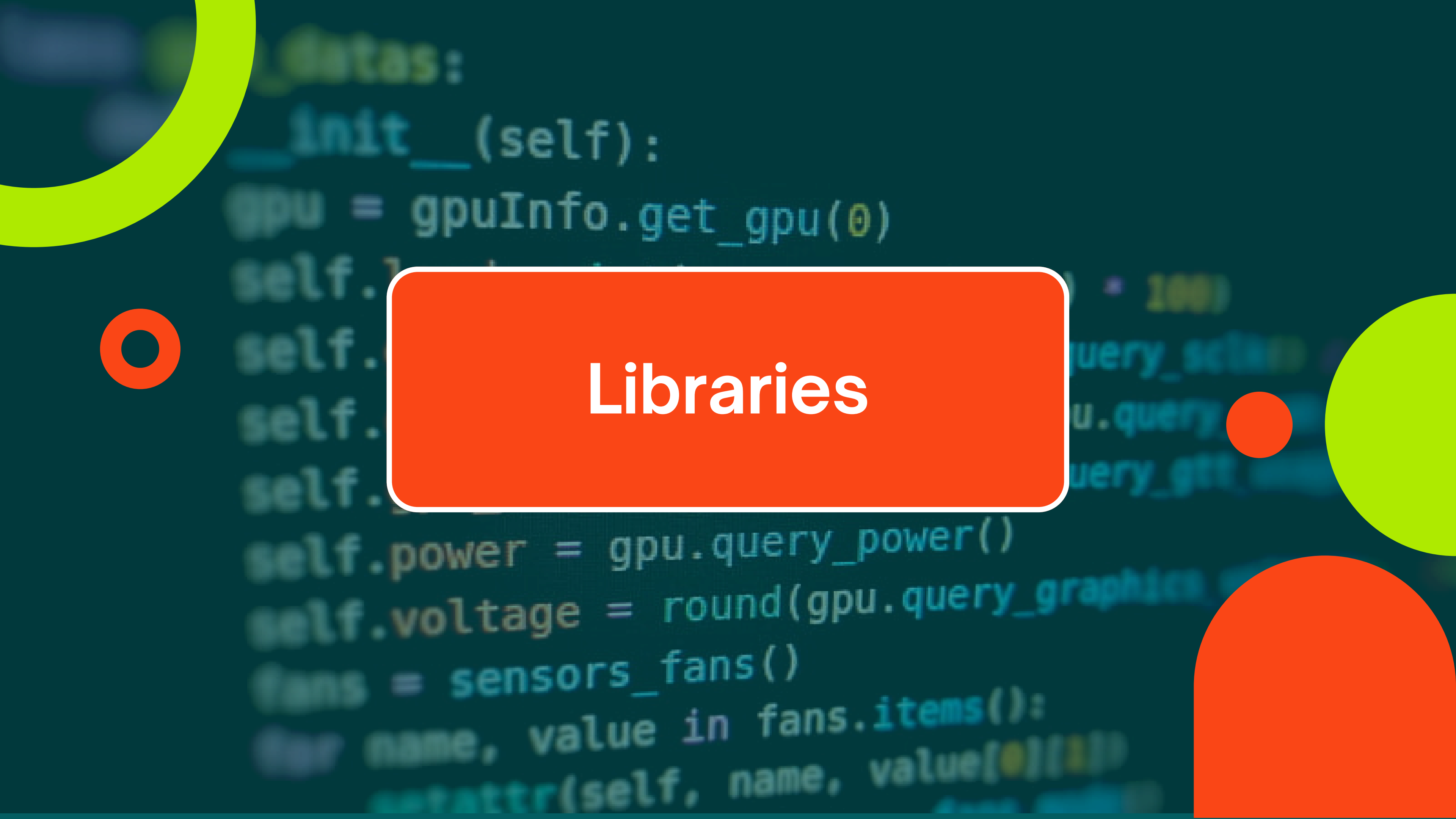
Import your module: In another Python script, you can import the module using the import keyword and **access its functions or variables**.

```
# main.py
import mymodule

print(mymodule.greet("Alice")) # Output: Hello, Alice!
print(mymodule.add(3, 4))      # Output: 7
```

✓ 0.0s

Reminder: Your custom module (mymodule.py) must be in the same directory as the files(main.py) to use it!

The background features a dark teal color with faint, blurred Python code snippets. Overlaid on this are several geometric shapes: a large lime green circle in the top-left, a smaller orange circle on the left, a large lime green circle on the right, and a large orange semi-circle in the bottom-right. A central orange rounded rectangle contains the title text.

Libraries

Using Pandas

- Using the **pandas library** in Python is a powerful way to read and manipulate data, including **CSV and JSON files**.
- **read_csv** for CSV files, **read_json** for JSON files
- Display data as a **DataFrame**

```
import pandas as pd

# Read the CSV file
df = pd.read_csv('sample_csv_students.csv')

# Display the DataFrame
print(df)
```

	Name	Age	Grade
0	Alice	21	A
1	Bob	22	B
2	Charlie	23	C

Application of Pandas

- Data Cleaning and Preparation
- Exploratory Data Analysis (EDA)
- Data Transformation
- File I/O Operations
- Integration with Other Libraries
 - Works well with **NumPy, Matplotlib, and Scikit-learn.**
 - Used in machine learning pipelines **for data preprocessing.**

Using NumPy (Numerical Python)

NumPy is a powerful Python library used for numerical and scientific computing.

```
import numpy as np

# Create a NumPy array
a = np.array([1, 2, 3, 4, 5])

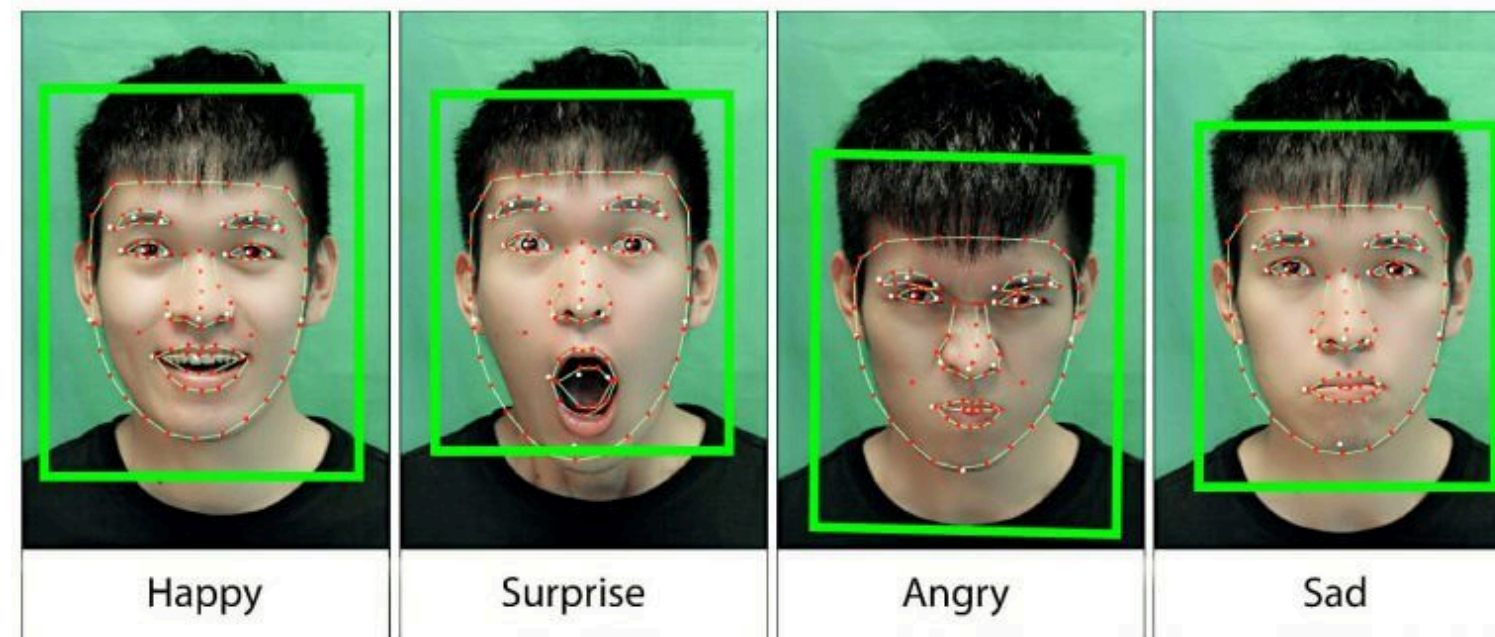
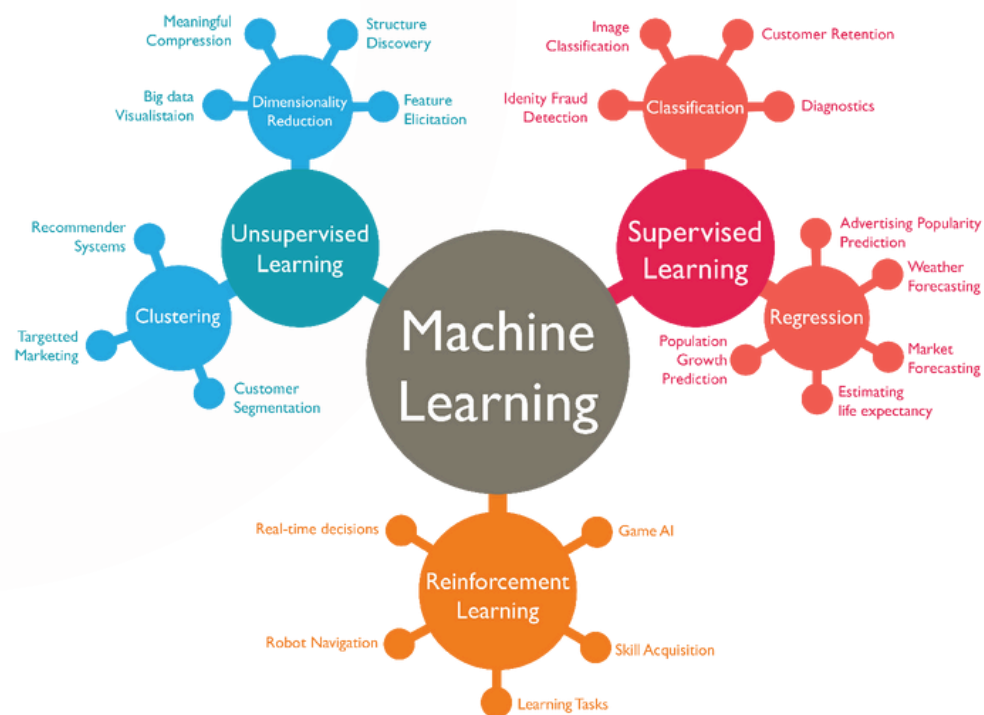
# Perform operations
print(a * 2) # [2 4 6 8 10]
print(np.mean(a)) # 3.0 (mean of the array)
```

✓ 0.6s

```
[ 2  4  6  8 10]
3.0
```

Applications of NumPy:

- Data analysis
- Machine learning
- Image processing
- Financial modeling
- Scientific research



Using Mathplotlib

Use for creating **static, interactive, and animated visualizations**. It is highly customizable and works seamlessly with other libraries like NumPy and Pandas, making it a powerful tool **for data visualization**.

```
import matplotlib.pyplot as plt

# Data
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

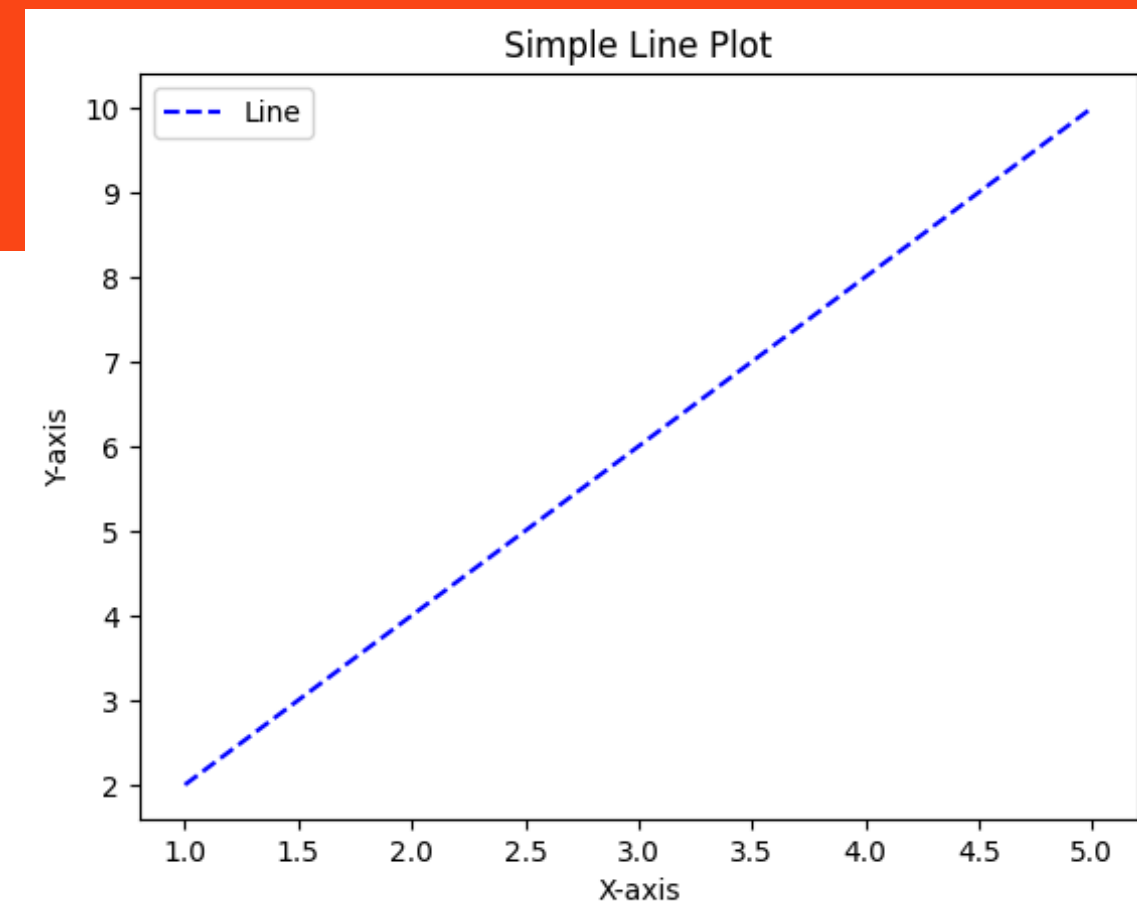
# Create a line plot
plt.plot(x, y, label='Line', color='blue', linestyle='--')

# Add title and labels
plt.title('Simple Line Plot')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')

# Add a legend
plt.legend()

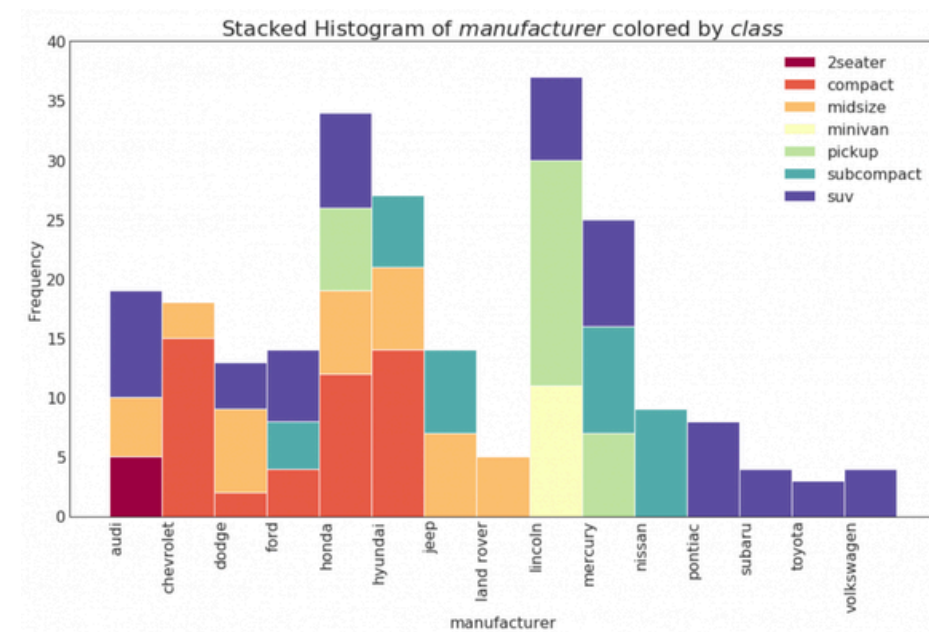
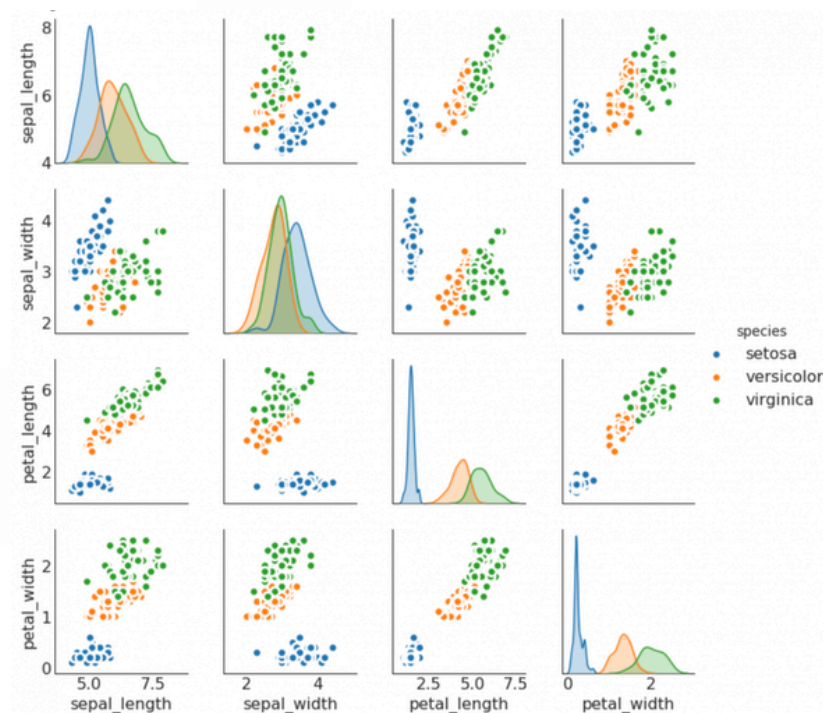
# Show the plot
plt.show()
```

✓ 2.9s



Application for Matplotlib

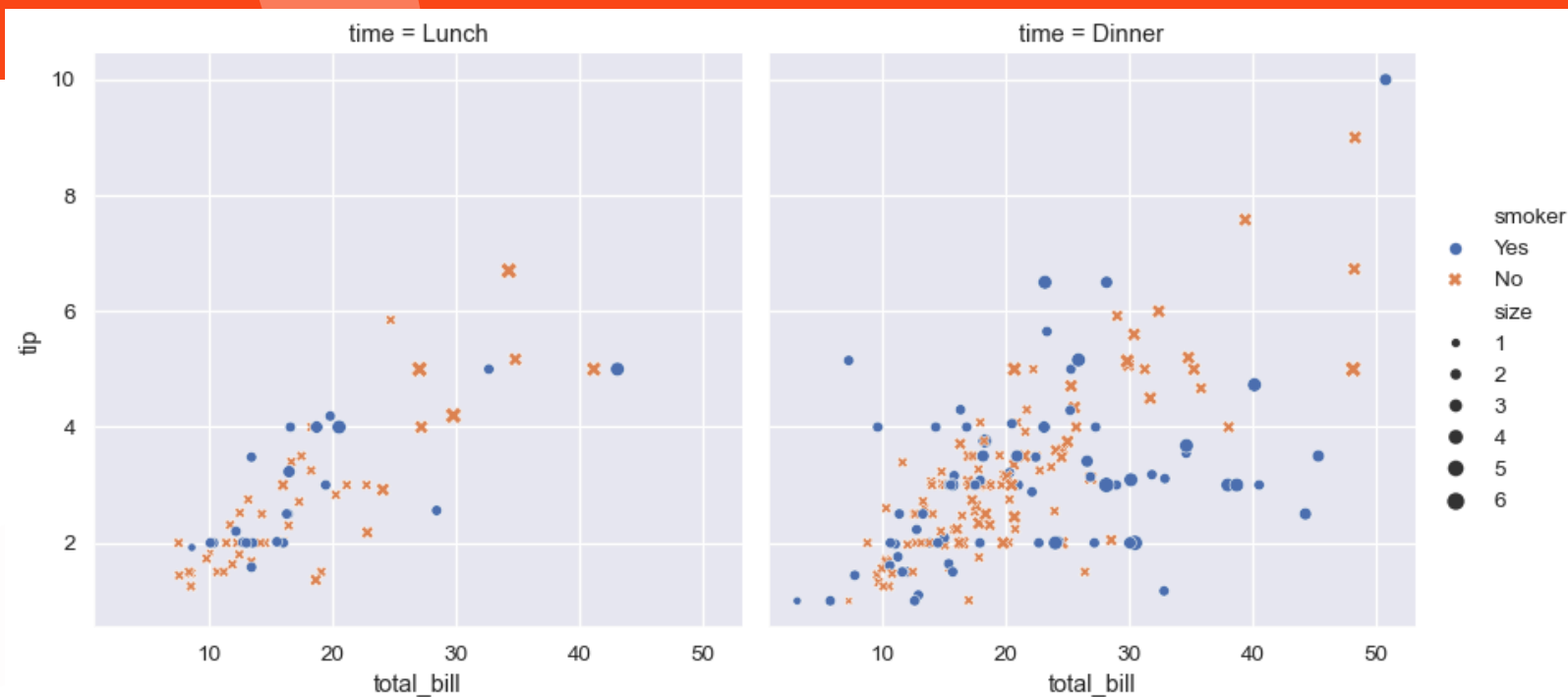
1. **Data Analysis:** Visualize trends and distributions.
2. **Machine Learning:** Plot model performance metrics (e.g., loss, accuracy).
3. **Scientific Research:** Represent experimental results and simulations.
4. **Finance:** Analyze stock prices and financial data.
5. **Engineering:** Visualize measurements, designs, and system behavior.



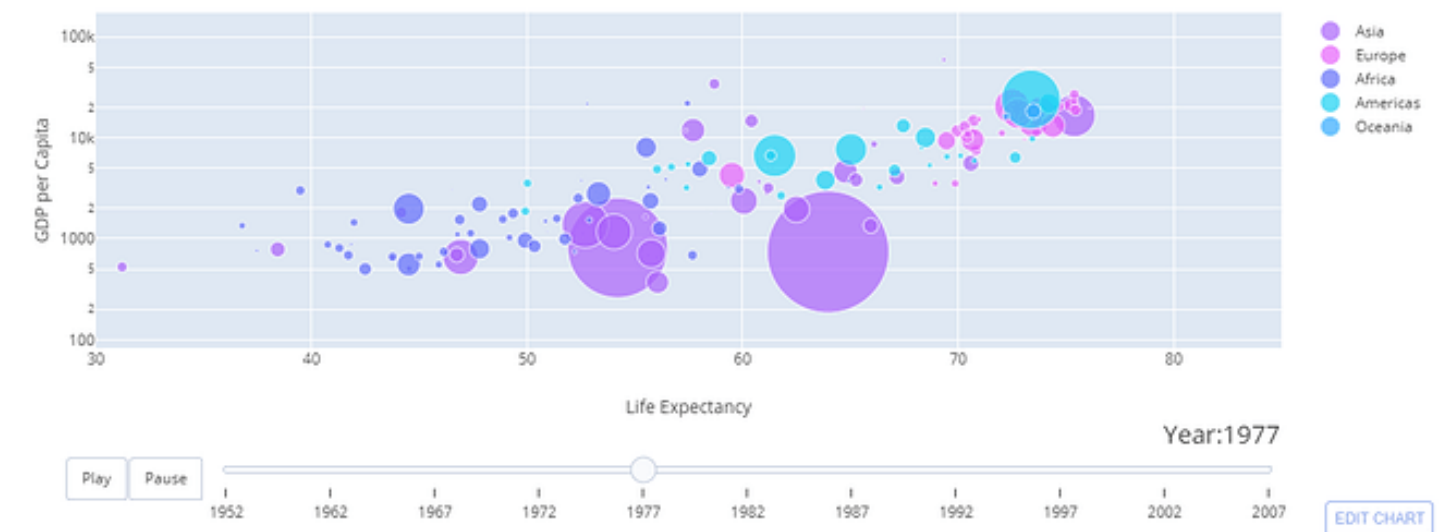
Other Library for Visualization

Seaborn: For advanced statistical plots

Plotly: For interactive visualizations



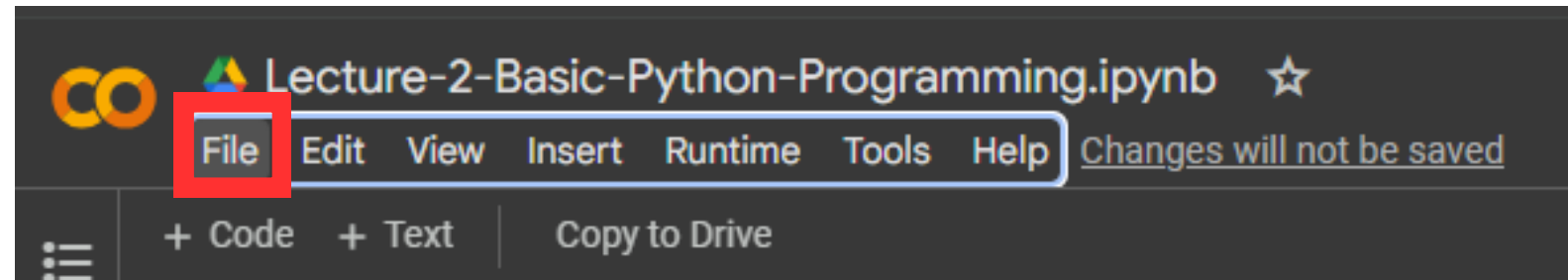
Scatter Plot in Seaborn



Scatter Plot in Plotly

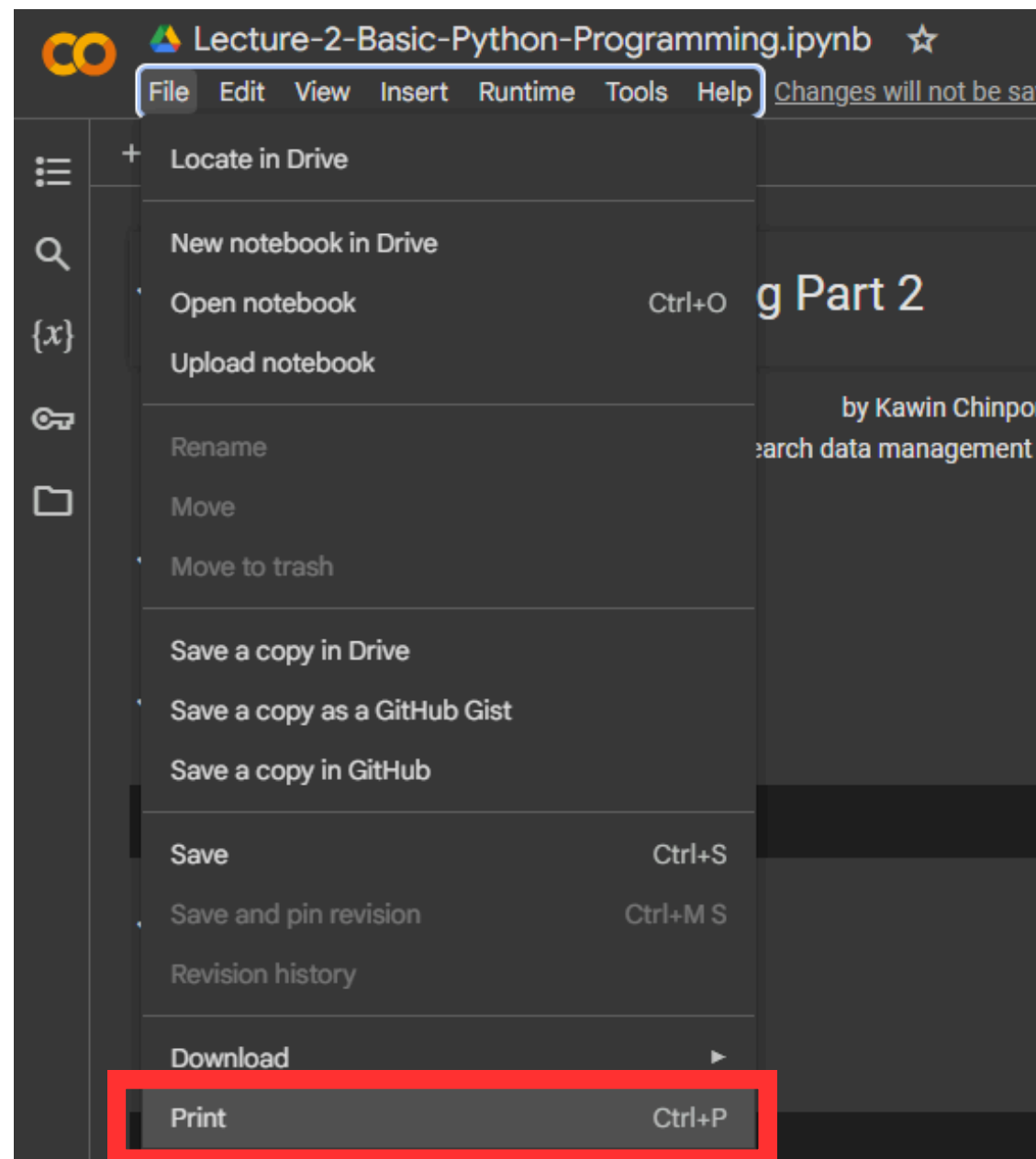
How to convert from .ipynb to .pdf in Google Colab

1.



: Click on “File”

2.



: Go to “Print”

How to convert from .ipnyb to .pdf in Google Colab

3.

1/21/25, 2:21 PM Lecture-2-Basic-Python-Programming.ipynb - Colab

Basic Python Programming Part 2

by Kavin Chinpong & Prachaya Khomduan
Research data management officer at Data management unit office

[1] Numpy

[1.0] import numpy library

```
import numpy as np
```

[1.1] ndarray initialization

Construct using python list

```
# 1d ndarray from 1d python list
list_a1=[1,2,3,5]
arr_a1=np.array(list_a1)
arr_a1

# 2d ndarray from 2d python list (list of list)
list_a2=[[1,2],[3,4],[5,6]]
arr_a2=np.array(list_a2)
arr_a2

list_a3=[[1,2],[2,3]],[[3,4],[4,5]]
arr_a3=np.array(list_a3)
arr_a3
```

or construct using some numpy classes and functions

```
np.zeros(5)

np.ones((3,4),dtype=float)

np.full((4,),999)

np.arange(3,10,2)

np.linspace(10,15,11)

np.random.choice(['a','b'],9)

np.random.randn(10)
```

[1.2] ndarray properties

```
list_a=[[1,2,3,4],[5,6,7,8],[9,10,11,12]]
arr_a=np.array(list_a)
arr_a

arr_a.ndim

arr_a.shape
```

https://colab.research.google.com/drive/1CA6zEToqlOKa3dgGvrb_j8K4WmD-DZT?usp=sharing#scrollTo=ofg5OJb75CJU&printMode=true

1/24

Print

24 pages

Destination

Save as PDF

Pages

All

Layout

Portrait

More settings

Save

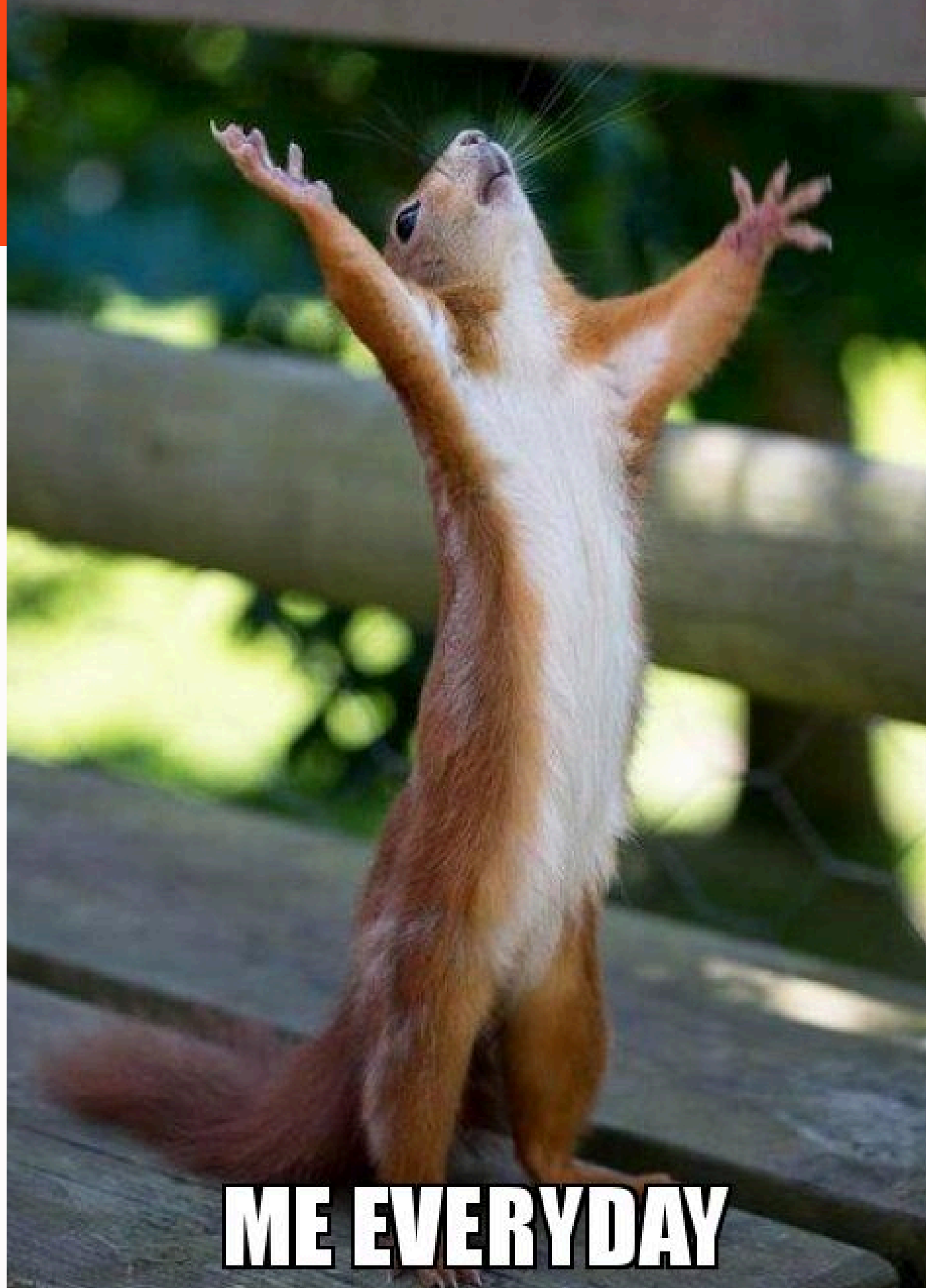
Cancel

: Select Destination to
“Save as PDF”. Then click
“Save”

Summary

- Basic Python Language
- File Handling
- How to use modules and import libraries

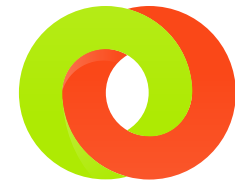
WHEN YOU FINISH CLASS



ME EVERYDAY

References

- Sansiri Tarn, Ph.D. (n.d.). CPE 102: Computer Programming Basics [Lecture notes], KMUTT.
- W3Schools. (n.d.). Python tutorial. Retrieved January 21, 2025, from <https://www.w3schools.com/python/default.asp>



CPE232
Data Models



THANK YOU

Dr. SANSIRI TARNPRADAB
WARAPOL PRATUMTA
SINSORN CHAITHAVORNKIT